InvoicePilot: Product Requirements Document

PayPal AI Hackathon, individual entry. Drafted Sat Nov 7, 2026. Deadline: Thu Nov 12, 12:00 pm PST (Fri Nov 13, 1:30 AM IST).

1. Summary

InvoicePilot is a chat-based billing agent for freelancers. The user types a message like 'Invoice Rahul $300 for logo design'. A Gemini-powered LangChain agent turns it into a draft PayPal invoice, and nothing is created or sent until the user clicks Approve. The agent can also check unpaid invoices and propose payment reminders, each of which also needs approval.

Pitch line: Freelancers lose hours writing invoices and chasing late payers. InvoicePilot turns one chat message into a PayPal invoice and follows up on unpaid ones, but never acts without the freelancer's approval.

2. Problem and audience

- **Audience:** independent freelancers who invoice international clients, for example designers, developers and writers.
- **Problem:** writing invoices is repetitive, and chasing late payments is awkward and easy to forget.
- **Why an agent:** it understands free-text requests, decides which invoices need a nudge, and uses PayPal tools to act.
- **Why approval-gated:** money actions should stay under human control. This is the agentic commerce angle for the Best Use of Agentic Commerce and Best Use of PayPal + AI prizes.

3. Goals and non-goals

Goals

- A working end-to-end flow in the PayPal sandbox: chat, draft, approve, invoice created and sent, buyer pays, status visible.
- A safe agent: destructive tools are not exposed, spending is capped, every action is logged.
- A clear 3-minute demo video and a README judges can follow in about 5 minutes.

Non-goals

- Autonomous or scheduled actions, multi-currency, INR, recurring invoices, refunds, subscriptions, user accounts, a database, mobile app.

4. Hackathon requirements checklist

- Project must integrate the PayPal developer platform (sandbox) and an AI tool, with PayPal central. Done by design.
- Public GitHub repo with an open-source license (MIT) visible in the repo About section.
- Working demo: README with complete setup and run instructions (no hosted demo required).
- Text description of features and functionality.
- Demo video under 3 minutes, public on YouTube, showing the project working, no copyrighted music or third-party trademarks.
- All materials in English.
- No secrets committed: only a `.env.example` is in the repo.
- Judging criteria (equal weight): Technological Implementation, Design, Potential Impact, Innovation, Presentation.

5. Features

P0 (must ship)

1. Chat to draft invoice: the agent extracts client, amount, description and email into a structured draft.
2. Approval card: shows client, email, amount (USD), description. Buttons: Approve, Reject.
3. On Approve: `create_invoice` then `send_invoice`.
4. List and summarise invoices: unpaid, overdue, paid.
5. Reminder on request: 'Remind Rahul' produces a proposed reminder, then Approve calls `send_invoice_reminder`.
6. Safety rails and an audit log (see section 8).

P1 (if time) 7. Check-in: 'Check my unpaid invoices' lists overdue ones and proposes a reminder for each, each needing Approve.

P2 (cut unless ahead on Tue Nov 10) 8. Earnings dashboard using `get_merchant_insights` or `list_transactions`.

6. User flows

Flow A: create an invoice

1. User: 'Invoice Rahul $300 for logo design.'
2. Agent looks up Rahul in `clients.json`. If missing, it asks for the email.
3. Agent returns a structured draft and the UI renders the approval card.
4. User clicks Approve. App code calls `create_invoice` and `send_invoice`. UI shows the invoice number and status.
5. Sandbox buyer opens the invoice email or link and pays.

Flow B: check and chase

1. User: 'Check my unpaid invoices.'
2. Agent calls `list_invoices`, summarises, and proposes a reminder for each overdue one.
3. User approves or rejects each. Approved ones call `send_invoice_reminder`.

7. Architecture and stack

- **UI:** Streamlit single app (chat plus approval cards plus audit log). Session state holds conversation and pending drafts.
- **Agent:** `agent.py` using `from langchain.agents import create_agent` with `ChatGoogleGenerativeAI`. Model name comes from `.env` (`GEMINI_MODEL`) so it can be swapped in one line.
- **PayPal:** `paypal-agent-toolkit` configured with the invoice tools. PayPal is the source of truth, so no database.
- **Config:** `.env` for PayPal client ID and secret (sandbox), `GOOGLE_API_KEY`, `GEMINI_MODEL`. `clients.json` maps client names to emails.
- **Pattern:** the LLM can call read tools freely. Write actions are returned as proposals, and only the Approve button executes them in app code.
- **Optional later:** a thin FastAPI wrapper around `agent.py`. Not needed for the hackathon.

8. Tool whitelist and safety rails

Read-only, free to call: `list_invoices`, `get_invoice`, and `get_merchant_insights` (P2).

Gated, only after Approve: `create_invoice`, `send_invoice`, `send_invoice_reminder`.

Never exposed: `delete_invoice`, `cancel_sent_invoice`, `record_refund_for_invoice`, `record_payment_for_invoice`, and all order and subscription tools.

Rails

- Maximum invoice amount: $1,000 (hard block with a clear message).
- Currency fixed to USD.
- The client email is always shown on the approval card, and invoices go only to emails confirmed by the user.
- Every proposed and executed action is written to an on-screen audit log.

9. Draft system prompt

You are InvoicePilot, a billing assistant for a freelancer. You can read invoices and propose new invoices or reminders. You never create, send or remind directly: always return a proposal and wait for the user to approve it. Use USD only. Never propose an invoice over $1,000. If the client's email is not known, ask for it before proposing. Never invent amounts, clients or invoice numbers. When summarising invoices, report only what the tools returned. If a request is outside invoicing (refunds, deleting, cancelling), say you can't do that here.

10. Demo script (under 3 minutes)

1. 0:00-0:15 Problem and pitch line.
2. 0:15-1:00 Prompt 1: 'Invoice Rahul $300 for logo design.' Show the draft card, click Approve, show the invoice in the sandbox dashboard.
3. 1:00-1:30 Switch to the sandbox buyer and pay the invoice.
4. 1:30-2:15 Prompt 2: 'Check my unpaid invoices.' Show the summary and a proposed reminder, then Approve.
5. 2:15-2:40 Prompt 3: 'Invoice Rahul $5,000.' Show the $1,000 cap blocking it.
6. 2:40-3:00 Close: what it is, who it is for, why approval matters.

Record in English with voiceover using OBS or Loom. No music. Upload as a public YouTube video.

11. Error cases to handle

- Unknown client, so ask for the email.
- Amount missing, zero, negative or over the cap.

- PayPal API error (auth, invoicing not enabled, validation): show a readable message and log it.
- Gemini model error or rate limit: show a retry message and never crash the UI.
- Double-click on Approve: disable the button after the first click.
- Prompt injection inside invoice descriptions: the tool whitelist limits the damage.

12. Schedule (about 11-12 hours total)

- **Sat Nov 7, 3h:** spike. Watch Tools, Tool Calling and Agent videos at speed. Gemini calls `list_invoices` and drafts `create_invoice` through the toolkit.
- **Sun Nov 8, 1-1.5h:** draft-and-confirm logic working in a plain script.
- **Mon Nov 9, 3-4h:** Streamlit UI, Approve and Reject buttons, audit log.
- **Tue Nov 10, 1-1.5h:** reminders, safety rails, error handling.
- **Wed Nov 11, 1-1.5h:** README, MIT license, `.env.example`, record the video.
- **Thu Nov 12, 1-1.5h:** upload to YouTube, submit by the evening IST.

13. Cut order if behind

1. Earnings dashboard (P2).
2. Proactive check-in (P1), keep 'Remind Rahul'.
3. UI polish.

Never cut: the approval gate, the README, the video.

14. Risks

- **Toolkit version conflicts** with current LangChain. Mitigation: do the spike first and pin versions if pip complains.
- **Gemini model availability.** Sources conflict about gemini-2.5-flash retirement. Mitigation: model name in `.env`, test early.
- **Invoicing not enabled** on the sandbox app or seller account. Mitigation: check in the spike on Day 1.
- **Time.** About 12 hours total for a first agent project. Mitigation: small scope and the cut order above.
- **Secrets leak.** Mitigation: `.gitignore` for `.env`, commit `.env.example` only.

15. Submission checklist

- [ ] Public GitHub repo with MIT license visible
- [ ] README with setup steps, required keys, and how to run
- [ ] `.env.example` and `clients.json.example` committed, no real secrets
- [ ] Text description of features and functionality
- [ ] Demo video under 3 minutes, public on YouTube
- [ ] Project submitted on Devpost before Thu Nov 12, 12:00 pm PST (1:30 AM IST Nov 13), aiming for the evening of Nov 12 IST

16. How this maps to the judging criteria

- **Technological Implementation:** LangChain agent, Gemini tool calling, PayPal toolkit invoice tools, proposal and approval pattern.
- **Design:** one clean Streamlit app with approval cards and an audit log.
- **Potential Impact:** a specific pain for a specific audience, shown working end to end.
- **Innovation:** an agent that acts but never moves money without a human click, with destructive tools deliberately hidden.
- **Presentation:** a tight video that shows the full create, pay and chase loop.